

Assignment 1: Byte Pair Encoding from Scratch

1. Goal

Your group writes a Byte Pair Encoding (BPE) tokenizer in Python, from scratch. You train it on a Shakespeare corpus. You then measure how well it works.

A character tokenizer has a small vocabulary but makes long sequences. A word tokenizer makes short sequences but fails on new words. BPE starts from characters and joins frequent pairs of symbols, one pair at a time. This assignment asks you to build that process and to show what it costs and what it gives.

2. Rules

1. Write the algorithm yourselves. Use only the Python standard library for it.
2. Do not use a tokenizer library for the algorithm. This rule covers `sentencepiece`, `tokenizers`, `transformers`, `tiktoken` and `subword-nmt`.
3. You may use `numpy`, `pandas` and `matplotlib` for tables and plots.
4. Make the run deterministic. Two runs on the same input must give the same merges.
5. Every member must be able to explain every part of the code. We ask in the review session.
6. You may use an AI assistant. If you do, write which parts you used it for.

3. Data

Use the Shakespeare corpus. The *tiny Shakespeare* file is enough (about 1.1 MB): <https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

Split it as follows. Read the file as UTF-8 text and split it into lines. Keep the last 1 % of the lines as the test set. Keep the other 99 % as the training set. Do not shuffle. Print the number of lines, words and characters for each part.

Choose two more texts for comparison. Do not train on them. Each text needs 200 kB or more. Take one text from each group:

- **Group A, modern English prose.** For example *Pride and Prejudice* or a set of news articles.
- **Group B, text that is not literary English.** For example a Python source file, a raw Wikipedia page, or a text in another language that uses the Latin alphabet.

4. Task 1: Prepare the data (15 points)

Write a function `normalize(text, strategy)`. Implement these four strategies:

Name	What it does
<code>raw</code>	It changes nothing.
<code>lower</code>	It makes all letters lowercase.
<code>clean</code>	It applies <code>lower</code> , then Unicode NFKC. It replaces each digit with 0. It collapses each run of spaces into one space.
<code>split</code>	It applies <code>clean</code> . It also puts a space around each punctuation mark.

BPE must not join symbols across a word boundary. Add the marker `</w>` to the end of each word, or add a marker character to the start of each word. Choose one for the main experiments. Implement the other one too, and compare the two at one value of k .

5. Task 2: Train the merges (30 points)

1. Count the word types in the training text. Keep a dictionary from a word to its count. Work on the types, not on the full text. This step makes the trainer much faster.

2. Split each word type into characters. Add the word boundary marker.
3. Count each pair of adjacent symbols. Weight each count by the count of the word.
4. Find the pair with the highest count. If two pairs tie, take the pair that is first in alphabetical order. This rule keeps the run deterministic.
5. Replace that pair with one new symbol in every word type.
6. Add the pair to the merge list. Add the new symbol to the vocabulary.
7. Repeat steps 3 to 6 until you reach k merges.

Speed. A simple trainer counts all pairs again after every merge. This is correct but slow. Your trainer must finish $k = 8000$ in less than 10 minutes. To reach this, keep the pair counts between the steps. After a merge, update only the word types that contain the merged pair. Report the training time for each value of k .

6. Task 3: Encode and decode (20 points)

1. Write `encode(text)`. It applies the merge rules to each word, in the learned order.
2. Write `decode(tokens)`. It returns a text string.
3. Map a character that is not in the training vocabulary to `<unk>`. Count these cases.
4. Test the round trip. For the `raw` strategy, `decode(encode(t))` must equal `t` for every line of the test set. Show this test in the notebook.
5. The other three strategies lose information. For those, compare against the *normalized* text, not against the original text.

Required interface. The notebook is the only thing that we run. You may keep the code in the notebook, or in `.py` files next to it. The `.py` files must import into the notebook without an error and without a path change. Use these names:

```
from bpe import BPETokenizer, normalize

tok = BPETokenizer(num_merges=1000, end_of_word="</w>")
tok.train(train_text)
ids = tok.encode("Shall I compare thee to a summer's day?")
text = tok.decode(ids)
tok.save("models/bpe_1000.json")
tok2 = BPETokenizer.load("models/bpe_1000.json")

tok.merges      # list of (str, str), in the order that the trainer learned them
tok.vocab       # dict from token string to token id
```

7. Task 4: Run the experiments (20 points)

Put each result in a table. Give every plot an axis label and a legend.

Experiment 1, vary k . Train on the training set with the `clean` strategy. Use these values:

$$k \in \{0, 100, 250, 500, 1000, 2000, 4000, 8000\}$$

Here $k = 0$ gives the character tokenizer. Add a word tokenizer as a second baseline; it keeps the 8000 most frequent words and maps the rest to `<unk>`. Report every measure of Task 5, for the training set and the test set. Plot the compression ratio against k , and the tokens per word against k . Use a logarithmic x axis.

Experiment 2, vary the normalization. Fix $k = 2000$. Train one tokenizer for each of the four strategies. Report the same measures. Add a row for the second word boundary marker.

Experiment 3, use a different kind of text. Take the tokenizers from Experiment 1. Do not train them again. Measure them on the Shakespeare test set and on your two comparison corpora. Put the three corpora in one table. Plot the tokens per word against k , with one line for each corpus.

8. Task 5: Measure the results (15 points)

Report these three measures:

$$\text{compression ratio} = \frac{\text{characters in the normalized text}}{\text{BPE tokens}} \quad \text{tokens per word} = \frac{\text{BPE tokens}}{\text{words}}$$

$$\text{UNK rate} = \frac{\text{number of <unk> tokens}}{\text{number of tokens}}$$

A word here is one item after a split on whitespace. Do not count the word boundary marker as a character. Write your counting rule in the notebook and keep the same rule everywhere. Report the UNK rate for the BPE tokenizer and for the word tokenizer, on all three corpora.

Accuracy. A tokenizer has no single natural accuracy. So you define one. Do both steps.

Step 1, the required check. Report the round-trip accuracy: the number of test lines where `decode(encode(line))` returns the input, divided by the number of test lines. The value must be 1.000. A lower value means a bug.

Step 2, your own measure. Choose one measure and define it with a formula. Say what a high value means, and say what it does not mean. Three suggestions:

- **Boundary agreement.** Write 60 English words by hand. Mark the morpheme boundaries yourselves, for example `un + happi + ness`. A word of L characters has $L - 1$ cut positions. Let S be the positions that BPE cuts and G the positions that you cut. Then $P = |S \cap G|/|S|$, $R = |S \cap G|/|G|$ and $F_1 = 2PR/(P + R)$. Sum the counts over all 60 words before you divide. Plot F_1 against k .
- **Stability.** Train two tokenizers on two halves of the training set with the same k . Measure how often they cut a test word at the same positions.
- **Consistency.** Measure how often the same word type gets the same segmentation in different corpora.

State the weakness of your measure. For example, BPE does not try to find morphemes, so a low F_1 against a morpheme list is not automatically a fault of your code.

9. What you hand in

Make one ZIP file with the name `group_NN_a1.zip`. Upload it to the folder of your group on StudIP. One member uploads for the whole group. The ZIP contains:

- `assignment1.ipynb`, with all output cells kept. It must run from top to bottom after *Restart and Run All*.
- Your `.py` files, if you use them, and `requirements.txt`.
- The saved merge files under `models/`. Keep the ZIP under 10 MB.
- `README.md`. See the next paragraph.

The contribution statement. The `README.md` must start with a table in this form:

Name	Student number	Tasks	Share
Student 1	123456	Tasks 2 and 3, encoder tests	40 %
Student 2	234567	Task 1, Task 5, plots	35 %
Student 3	345678	Task 4, comparison corpora, README	25 %

The shares must sum to 100. All members must agree to the table. Add one sentence about the work of each member in your own words. We grade the submission as one piece of work, but we use this table in the review session, and we can adjust a single grade if the table and the review do not agree. A submission without this table loses 10 points.

The `README.md` also gives the total run time, the machine that you used, the sources of your two comparison corpora, and your statement about AI use.

Do not hand in the corpora. Do not hand in empty output cells. If the full run takes more than 15 minutes, cache the models to `models/` and read them back. In that case add a flag `FORCE_RETRAIN = False` at the top of the notebook.

10. Grades

Item	Points
Task 1. Data split and the four normalization strategies	15
Task 2. Correct and deterministic merge training, with the speed requirement	30
Task 3. Encoder, decoder, exact round trip, required interface	20
Task 4. The three experiments, with tables and plots	20
Task 5. Correct measures and your own accuracy measure	15
Total	100

We remove points for these problems: the notebook does not run from top to bottom, -15 ; the `.py` files do not import, -10 ; the contribution table is absent, -10 ; the round trip is not exact and you do not say so, -10 ; two runs give different merges, -5 ; a plot has no axis label or no legend, -2 each.

11. The next assignment

Assignment 2 uses the output of this one. You build n -gram language models on top of your BPE tokens. You then compare the perplexity for different values of n and of k . The choice of k has a direct effect on perplexity, because perplexity depends on the unit that you count. Keep your merge files and your tokenizer code. You will import them again.